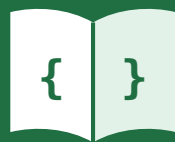

Module 06 — Spring Data JPA and Hibernate

Exercises

Spring Boot Edu



2026-09-23

Contents

How to Work	2
Exercise 1 — Mapping the Order Aggregate (Easy)	2
Exercise 2 — Finding and Fixing N+1 (Medium)	3
Exercise 3 — Dynamic Order Search (Hard)	3
Bonus Challenge (Optional)	4

How to Work

1. The starter code is in `modules/06-data-jpa-postgres/exercise/`. Find the TODO comments.
2. The tests run against a real PostgreSQL (Testcontainers). **Docker must be running.**
3. Run the tests:

```
./mvnw -Pexercises -pl modules/06-data-jpa-postgres/exercise -am test
```

4. The exercise is done when all tests are green.
5. If you get stuck, read the hints first, then the solution in `modules/06-data-jpa-postgres/solution/`.

! Important

Do the exercises in order. Exercises 2 and 3 need the order lines you map in Exercise

1. The tables and sample data (V1, V2) are provided.

Exercise 1 — Mapping the Order Aggregate (Easy)

Goal: map a bidirectional 1-N relationship correctly, with cascade and orphan removal.

An order (`PurchaseOrder`) must be saved and deleted together with its lines (`OrderLine`). Lines are added and removed only through the order.

Tasks (package `exercisel`):

- TODO 1a — `PurchaseOrder.lines`: a 1-N mapping instead of `@Transient`. The relationship is owned by `OrderLine.order`. Lines are saved together with the order. A line removed from the list is deleted from the database.
- TODO 1b — `OrderLine.order`: a lazy N-1 mapping to the `order_id` column instead of `@Transient`.
- TODO 1c — `addLine` and `removeLine`: set up **both sides** of the relationship together.

Hints:

- Lesson section 3.1: `@OneToMany(mappedBy = ..., cascade = ..., orphanRemoval = ...)`, `@ManyToOne(fetch = FetchType.LAZY)`, `@JoinColumn`.
- Create a new line with `new OrderLine(this, ...)`.

Acceptance criteria: all 4 tests in `ExerciselTest` pass.

Estimated time: 30 minutes

Exercise 2 — Finding and Fixing N+1 (Medium)

Goal: prove an N+1 problem by counting SQL statements, fix it, and let the database compute a report.

`findByCustomerEmail` returns a customer's orders. Touching every order's lines to compute totals runs $1 + N$ queries. The test measures this (`theProblem_oneQueryPerOrder` is already green).

Tasks (`exercise2.OrderRepository`):

- TODO 2a — `findWithLinesByCustomerEmail`: load the orders **and** their lines with a single SQL statement.
- TODO 2b — `totalsPerCustomer`: replace the default method with a JPQL query that returns one `CustomerTotal` per customer. `Total = quantity * unitPrice` over all lines. Highest total first, computed by the database.

Hints:

- Lesson section 3.3: `@EntityGraph(attributePaths = ...)`.
- Lesson section 3.4: `select new package.CustomerTotal(...) ... group by ... order by ....`

Acceptance criteria: all 3 tests in `Exercise2Test` pass (both solutions run **one** SQL statement).

Estimated time: 30 minutes

Exercise 3 — Dynamic Order Search (Hard)

Goal: build a flexible query from optional criteria with `Specification`.

Tasks (`package exercise3`):

- TODO 3a — `OrderSpecifications`: the customer, status and "created after" criteria.
- TODO 3b — `containsIsbn`: the order has at least one line with this ISBN. Each order appears **once** in the result.
- TODO 3c — `OrderSearch.search`: combine only the criteria that are set (all must match), query with paging, and turn every order into an `OrderSummary`.

Hints:

- Lesson section 3.5: `(root, query, cb) -> cb.equal(root.get("..."), ...), cb.greaterThan, root.join("lines").`
- Joining a collection can return the same order several times: `query.distinct(true).`
- `Specification.allOf(list)` and `repository.findAll(spec, pageable).`

Acceptance criteria: all 5 tests in `Exercise3Test` pass.

Estimated time: 45 minutes

Bonus Challenge (Optional)

Add `@Version` to `PurchaseOrder` (with a version column in a new Flyway migration). Write a test showing that one of two concurrent updates is rejected with an `ObjectOptimisticLockingFailureException` (lesson section 3.6).